A Rainbow DQN Approach for Air Hockey

Lalit Chaudhary (zxold61-rainbow)

Abstract

We present a systematic study of Deep Reinforcement Learning for air hockey in a physics-based simulation. We implement multiple DQN variants and a Rainbow agent, analyzing the contributions of Double Q-learning, Prioritized Experience Replay, Dueling Networks, Multi-step returns, Distributional RL (C51), and Noisy Networks. Contrary to Atari results, Noisy Networks significantly degrade performance in this low-dimensional adversarial setting, causing a 42 percentage point drop in win rate. Our ablation study reveals that removing Noisy Networks and using 3-step returns yields the best configuration. When combined with alternating opponent training, this achieves near-100% win rate against both weak and strong rule-based opponents after 200,000 episodes. [1]

1 Introduction

Deep Reinforcement Learning (DRL) has achieved remarkable results in sequential decision-making, from Atari games [1] to Go [2]. Competitive environments remain challenging due to the need to adapt to adversarial behavior.

Air hockey provides a controlled testbed requiring real-time control, precision, and opponent modeling. Unlike single-agent tasks, performance depends on anticipating and countering adversarial strategies.

Rainbow DQN [3] combines six improvements to DQN, each targeting complementary learning challenges. However, the effectiveness of these components is domain-dependent. We investigate which Rainbow components transfer effectively to the air hockey setting and analyze opponent training strategies for improved generalization.

2 Environment Description

2.1 Hockey Environment

The Hockey environment simulates a two-player air hockey game implemented using the Box2D physics engine. Each player controls a paddle (racket) and attempts to score by hitting the puck into the opponent's goal. The simulation runs at 50 frames per second (FPS), with a maximum episode length of 250 timesteps in normal mode.

State Space: The observation space is an 18-dimensional continuous vector containing:

- Player 1: position (x, y) relative to center, angle, linear velocities (v_x, v_y), and angular velocity (6 dimensions)
- Player 2: same as Player 1 (6 dimensions)
- Puck: position (x, y) relative to center and linear velocities (v_x, v_y) (4 dimensions)
- Puck possession timers for both players (2 dimensions)

Action Space: The native action space is continuous, consisting of forces in the x and y directions, a torque for rotation, and optionally a shoot action (3–4 continuous values per player). For our DQN-based approach, we discretize this into 8 discrete actions using the environment's built-in discretization:

- Action 0: No movement (stay in place)
- Actions 1–4: movements in (up, down, left, right) direction
- Actions 5–6: Angle rotation (clockwise, anticlockwise)
- Action 7: Shoot (when in possession of the puck)

Reward Structure: The environment provides sparse terminal rewards based on game outcomes: $+10$ for scoring a goal (winning), -10 for conceding a goal (losing) and 0 for a draw (timeout). Additionally, the environment includes reward shaping based on proximity to the puck when it is in the agent's half and moving towards the agent's goal. This shaping reward encourages defensive positioning.

Game Modes: The environment supports three modes: `NORMAL` (standard two-player game for 250 timesteps), `TRAIN_SHOOTING` (training mode for offensive play for 80 timesteps) and `TRAIN_DEFENSE` (training mode for defensive play for 80 timesteps). All experiments in this work use the `NORMAL` mode.

2.2 Opponents

The environment provides two built-in rule-based opponents implemented via the `BasicOpponent` class: **Weak Opponent**, a basic rule-based agent with limited responsiveness and simpler decision-making, and **Strong Opponent**, an improved rule-based agent with better puck tracking, positioning, and shooting behavior. Both opponents use hand-crafted heuristics for movement and shooting decisions. The performance gap between them provides a useful benchmark for evaluating agent generalization.

[1] The code is made availavle at `https://github.com/lalit3c/RainbowDQN-AirHockey`

3 Background

3.1 Deep Q-Networks

The Deep Q-Network (DQN) algorithm [1] approximates the optimal action-value function $Q^*(s, a)$ using a neural network with parameters θ. The Q-function represents the expected cumulative discounted reward when taking action a in state s and following the optimal policy thereafter:

$$Q^*(s, a) = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s, a_0 = a\right] \tag{1}$$

where $\gamma \in [0, 1)$ is the discount factor. The network is trained to minimize the temporal difference (TD) error:

$$\mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\left[(y - Q(s, a; \theta))^2\right] \tag{2}$$

where the target y is computed as:

$$y = r + \gamma \max_{a'} Q(s', a'; \theta^-) \tag{3}$$

and θ^- represents the parameters of a periodically updated target network that stabilizes training.

3.2 Rainbow DQN Components

Rainbow DQN [3] combines six key improvements to the original DQN algorithm:

1. Double Q-learning [4]: Addresses overestimation bias by decoupling action selection from evaluation:

$$y = r + \gamma Q(s', \arg\max_{a'} Q(s', a'; \theta); \theta^-) \tag{4}$$

2. Prioritized Experience Replay (PER) [5]: Samples transitions with probability proportional to their TD error magnitude:

$$P(i) = \frac{p_i^\alpha}{\sum_k p_k^\alpha} \tag{5}$$

where $p_i = |\delta_i| + \epsilon$ is the priority based on TD error δ_i, and α controls the degree of prioritization.

3. Dueling Networks [6]: Decomposes Q-values into state value $V(s)$ and advantage $A(s, a)$:

$$Q(s, a) = V(s) + A(s, a) - \frac{1}{|\mathcal{A}|}\sum_{a'} A(s, a') \tag{6}$$

4. Multi-step Learning: Uses n-step returns for improved credit assignment:

$$G_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n \max_{a'} Q(s_{t+n}, a'; \theta^-) \tag{7}$$

5. Distributional RL (C51) [7]: Models the full distribution of returns rather than just the expectation using a categorical distribution over a fixed support:

$$Z(s, a) = \sum_{i=0}^{N-1} p_i(s, a)\delta_{z_i} \tag{8}$$

where $\{z_i\}_{i=0}^{N-1}$ is a fixed support over $[V_{min}, V_{max}]$ and $p_i(s, a)$ are the learned probabilities.

6. Noisy Networks [8]: Replaces ϵ-greedy exploration with parametric noise in network weights:

$$y = (\mu^b + \sigma^b \odot \epsilon^b) + (\mu^W + \sigma^W \odot \epsilon^W)x \tag{9}$$

where μ and σ are learnable parameters and ϵ is sampled noise.

4 Method

4.1 Agent Implementations

We implement two agent families: DQN variants and Rainbow DQN.

4.1.1 DQN Agent

We consider four DQN variants: **Standard** (target network + replay), **Double** (Double Q-learning), **Dueling** (separate value and advantage), and **Double-Dueling** (combination of both).

Architecture. The standard DQN uses a feedforward network: Linear(18, 256) → ReLU → Linear(256, 256) → ReLU → Linear(256, 8).

The dueling architecture separates value and advantage after the first hidden layer:

- Value stream: Linear(256, 256) → ReLU → Linear(256, 1)
- Advantage stream: Linear(256, 256) → ReLU → Linear(256, 8)

They are combined as: $Q(s, a) = V(s) + A(s, a) - \frac{1}{8}\sum_{a'} A(s, a')$.

4.1.2 Rainbow Agent

Our Rainbow implementation integrates all six components with a network architecture that supports both distributional outputs and the dueling structure:

Network Architecture:

- Feature extraction: Linear(18, 256) → ReLU
- Value stream: Linear/NoisyLinear(256, 256) → ReLU → Linear/NoisyLinear(256, N_{atoms})
- Advantage stream: Linear/NoisyLinear(256, 256) → ReLU → Linear/NoisyLinear(256, $8 \times N_{atoms}$)

where $N_{atoms} = 51$ for the C51 distribution. The outputs are combined using the dueling formula and then passed through a softmax over the atom dimension to produce probability distributions.

Noisy Linear Layers: When enabled, we use factorized Gaussian noise [8] with initial standard deviation $\sigma_0 = 0.5$:

$$\mu \sim \text{Uniform}(-1/\sqrt{n_{in}}, 1/\sqrt{n_{in}})$$
$$\sigma_{\text{weight}} = \sigma_0/\sqrt{n_{in}}, \quad \sigma_{\text{bias}} = \sigma_0/\sqrt{n_{out}} \tag{10}$$

where n_{in} and n_{out} are input and output dimensions.

4.2 Prioritized Experience Replay

We implement PER using a Sum Tree data structure for efficient $O(\log n)$ sampling. The buffer stores transitions $(s, a, r, s', \text{done})$ along with priorities.

Priority Computation: New transitions are inserted with maximum priority to ensure they are sampled at least once. After training, priorities are updated based on the cross-entropy loss:

$$p_i = (|\mathcal{L}_i| + \epsilon)^\alpha \tag{11}$$

where $\epsilon = 10^{-6}$ prevents zero probabilities and $\alpha = 0.5$ controls prioritization strength.

Importance Sampling: To correct for the bias introduced by non-uniform sampling:

$$w_i = \left(\frac{1}{N} \cdot \frac{1}{P(i)}\right)^\beta \tag{12}$$

where β is annealed from $\beta_{start} = 0.4$ to 1.0 over 100,000 frames.

4.3 N-step Returns

We compute n-step returns by accumulating transitions in a buffer:

$$G_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n Q(s_{t+n}, a^*; \theta^-) \tag{13}$$

where $a^* = \arg\max_{a'} Q(s_{t+n}, a'; \theta)$ (Double Q-learning). The n-step transition $(s_t, a_t, G_t^{(n)}, s_{t+n}, \text{done}_{t+n})$ is stored in the prioritized replay buffer. At episode boundaries, remaining transitions are processed with done = True.

4.4 Distributional Bellman Update

For C51, we project the target distribution onto the fixed support $\{z_i\}_{i=0}^{N-1}$ using categorical projection:

$$\hat{T}z_j = r + \gamma^n z_j \quad (\text{clipped to } [V_{min}, V_{max}]) \tag{14}$$

The projection distributes probability mass to neighboring atoms based on distance:

$$m_l \mathrel{+}= p_j(s', a^*) \cdot (u - b_j) \tag{15}$$
$$m_u \mathrel{+}= p_j(s', a^*) \cdot (b_j - l) \tag{16}$$

where $b_j = (\hat{T}z_j - V_{min})/\Delta z$, and $l = \lfloor b_j \rfloor$, $u = \lceil b_j \rceil$.

Loss Function: We minimize the cross-entropy between predicted and projected distributions:

$$\mathcal{L} = -\sum_{i=0}^{N-1} m_i \log p_i(s, a) \tag{17}$$

Gradients are clipped to a maximum norm of 10.0 to prevent instability.

4.5 Modifications for Air Hockey

Removal of Noisy Networks: Our ablation study (Section 5.1) showed that Noisy Networks degrade performance. We therefore replace noisy linear layers with standard linear layers. Exploration is provided through the prioritized replay mechanism and the stochasticity of the distributional value estimates during early training.

Opponent-Mixing Strategy: To improve generalization, we alternate between weak and strong opponents each episode during training.

4.6 Training Procedure

Training proceeds as follows for each episode:

1. Reset environment with selected opponent
2. For each timestep (up to 250):
   - Select action (ϵ-greedy for DQN, greedy for Rainbow)
   - Execute action and observe (r, s', done)
   - Store transition (with n-step processing for Rainbow)
   - Perform training step every 4 steps if buffer has sufficient samples
3. Decay ϵ (DQN only)
4. Evaluate every 100 episodes against both opponent types

Target Network Updates: We use both hard and soft updates:

- Hard update every 8,000 steps: $\theta^- \leftarrow \theta$
- Soft update at other steps: $\theta^- \leftarrow \tau\theta + (1-\tau)\theta^-$ with $\tau = 0.005$

Optimizer: We use Adam optimizer with learning rate 6.25×10^{-5} and $\epsilon_{Adam} = 1.5 \times 10^{-4}$.

5 Experiments

We conduct three sets of experiments: (1) an ablation study comparing agent architectures and Rainbow components, (2) an analysis of opponent training strategies, and (3) extended training of our best-performing configuration.

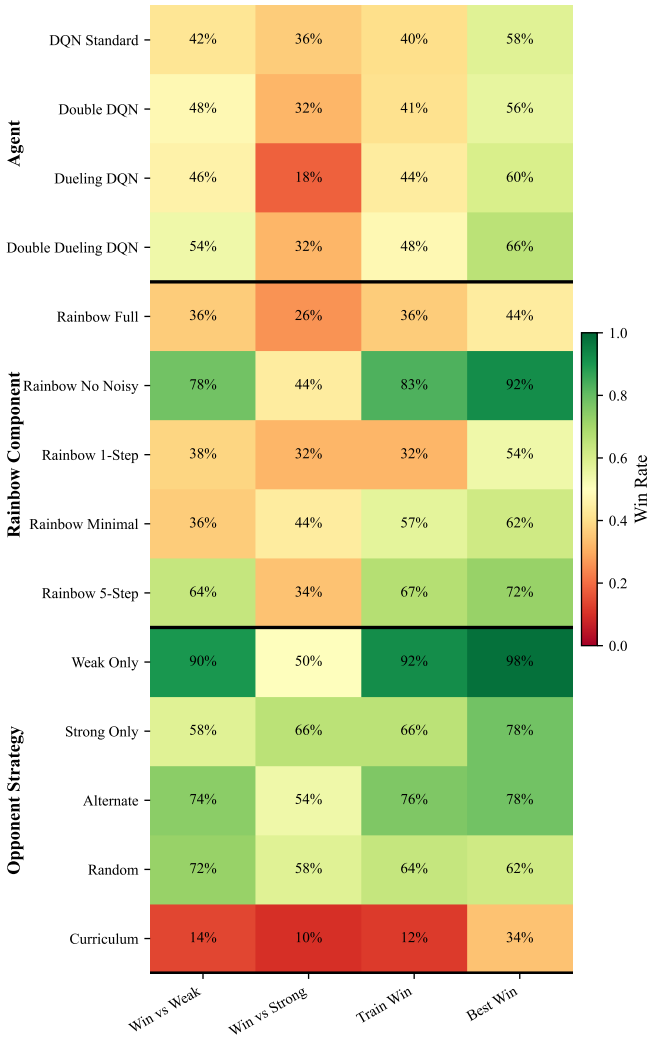

Figure 1: Ablation study results comparing DQN variants and training strategies in the hockey environment. Train Win indicates final training win rate while Best Win indicated highest win rate observed during evaluation. Rainbow Minimal refers to removal of noisy layers and using n_step = 1

5.1 Ablation Study

5.1.1 Experimental Setup

All ablations were trained for 10,000 episodes against the weak opponent (except opponent-strategy experiments which use the specified strategy). Evaluation was conducted every 100 episodes using 50 episodes each against weak and strong opponents. Figure 1 shows final evaluation win rates.

5.1.2 Agent Architecture Comparison

Full Rainbow underperforms all DQN variants (36% weak, 26% strong vs. 42–54% weak for DQN). Removing Noisy Networks yields the largest improvement: 36% $\rightarrow$ 78% against weak and 26% $\rightarrow$ 44% against strong. Multi-step returns are critical: $n = 1$ performs poorly (38% weak), while $n = 3$ and $n = 5$ achieve 78% and 64% respectively. Among DQN variants, Double-Dueling DQN performs best (54% weak, 32% strong).

5.1.3 Opponent Training Strategy Comparison

Using Rainbow without Noisy Networks ($n = 3$) as the base agent: weak-only training maximizes weak performance (90%) but generalizes poorly (50% strong). Strong-only achieves the best strong performance (66%) but reduces weak performance (58%). Alternating provides balanced results (74% weak, 54% strong). Random mixing (72% weak, 58% strong) is competitive with alternating. Curriculum learning fails catastrophically (14% weak, 10% strong), likely due to the non-smooth difficulty transition between opponents.

5.2 Extended Training

Based on the ablation study, we selected Rainbow without Noisy Networks ($n = 3$) with alternating opponent training and trained for 200,000 episodes. Figure 2 shows training progress. Episode rewards increase steadily with decreasing variance.

After 200,000 episodes, the agent achieves an approximately 100% win rate against both weak and strong opponents, with an average reward of 8.39 ± 4.86. This demonstrates that Rainbow without Noisy Networks, combined with alternating opponent training, yields robust generalization across opponent types.

6 Discussion

The negative impact of Noisy Networks contrasts sharply with their success in Atari [3]. This discrepancy likely stems from the low-dimensional state space (18 dimensions fully specify the game), the adversarial setting where stochastic behavior can be exploited, and the small action space where simple exploration suffices. Additionally, injected noise may interfere with learning the precise control required for air hockey.

Multi-step returns prove crucial because rewards are sparse and primarily terminal, making 1-step credit assignment ineffective. Larger n accelerates reward propagation, and the deterministic physics ensures stable bootstrapping.

Opponent selection induces a clear trade-off: weak-only training accelerates learning but limits robustness; strong-only improves strong performance but slows progress; mixed strategies balance both. Curriculum learning fails catastrophically (14% win rate), suggesting adversarial difficulty is qualitative rather than smoothly scalable—the behavioral gap between opponents prevents gradual adaptation.

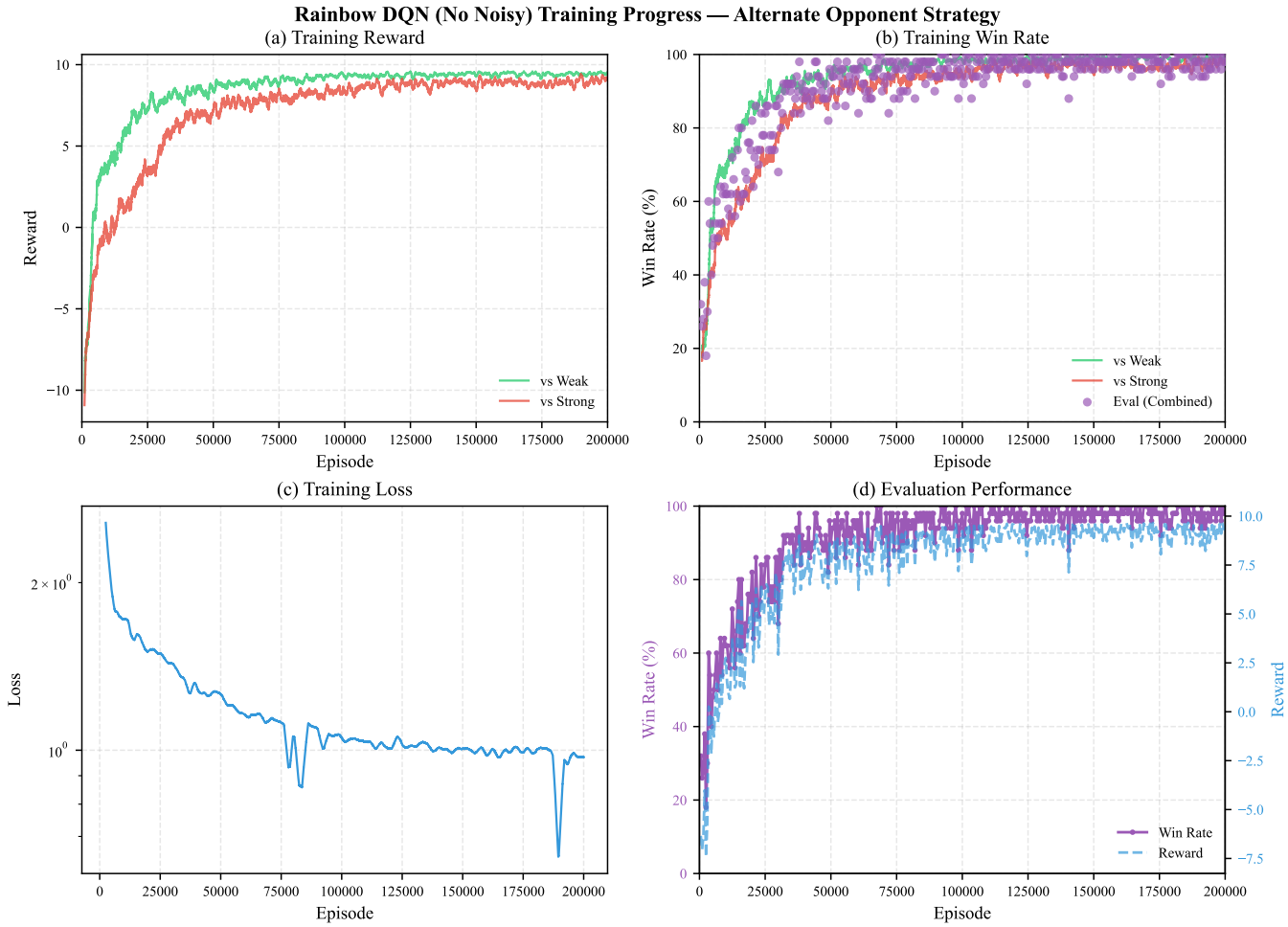

Figure 2: Training dynamics of Rainbow DQN (without Noisy Networks) under alternate opponent scheduling. (a) Smoothed training reward (rolling mean, window = 500 episodes) showing distinct learning trajectories against weak (green) and strong (red) opponents. (b) Training win rate (%) over episodes. Purple markers indicate periodic evaluation checkpoints (every 500 episodes) measuring combined performance against both opponent types. (c) Training loss displayed on a logarithmic scale (smoothing window = 2,500 steps) (d) Evaluation performance measured periodically: win rate (solid purple, left axis) and cumulative episode reward (dashed blue, right axis). All training curves use a rolling mean with window size of 500 episodes to reduce noise while preserving learning trends.

7 Conclusion

We conducted a systematic study of Rainbow DQN for air hockey, identifying which components transfer effectively to this adversarial low-dimensional setting. Contrary to Atari results, Noisy Networks significantly degrade performance (42 percentage point drop in win rate). Component selection is critical: Rainbow without Noisy Networks substantially outperforms both vanilla DQN and full Rainbow (78% vs. 42% and 36% against weak opponents). Multi-step returns ($n = 3$) are essential for credit assignment with sparse rewards. Opponent diversity improves generalization, with alternating strategies providing balanced performance across opponent types. After extended training (200,000 episodes), the final agent achieves near-100% win rates against both weak and strong opponents.

Hyperparameters

Table 1 summarizes the hyperparameters used in our final Rainbow (No Noisy) agent.

Table 1: Hyperparameters for Rainbow (No Noisy) Agent

Parameter	**Value**
Learning rate	6.25×10^{-5}
Adam ϵ	1.5×10^{-4}
Discount factor γ	0.99
Batch size	32
Replay buffer size	100,000
Target update (hard)	every 8,000 steps
Soft update τ	0.005
Update frequency	every 4 steps
N-step returns n	3
Number of atoms (C51)	51
V_{min}, V_{max}	-10.0, 10.0
PER α	0.5
PER β_{start}	0.4
PER β annealing	100,000 frames
Hidden dimensions	(256, 256)
Gradient clipping	max norm 10.0

Use of Large Language Models

The use of Large Language Models (LLMs) was limited to code autocompletion within the VS Code development environment. Additionally, portions of the visualization code for generating the ablation heatmap (Figure 1) and training progress plots (Figure 2) were developed with assistance from Claude LLM.

References

[1] Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.

[2] Silver, D., Schrittwieser, J., Simonyan, K., Antonoglou, I., Huang, A., Guez, A., Hubert, T., Baker, L., Lai, M., Bolton, A., et al. Mastering the game of Go without human knowledge. *Nature*, 550(7676):354–359, 2017.

[3] Hessel, M., Modayil, J., Van Hasselt, H., Schaul, T., Ostrovski, G., Dabney, W., Horgan, D., Piot, B., Azar, M., and Silver, D. Rainbow: Combining improvements in deep reinforcement learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 2018.

[4] Van Hasselt, H., Guez, A., and Silver, D. Deep reinforcement learning with double Q-learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 30, 2016.

[5] Schaul, T., Quan, J., Antonoglou, I., and Silver, D. Prioritized experience replay. In *International Conference on Learning Representations (ICLR)*, 2016.

[6] Wang, Z., Schaul, T., Hessel, M., Van Hasselt, H., Lanctot, M., and De Freitas, N. Dueling network architectures for deep reinforcement learning. In *International Conference on Machine Learning (ICML)*, pages 1995–2003. PMLR, 2016.

[7] Bellemare, M. G., Dabney, W., and Munos, R. A distributional perspective on reinforcement learning. In *International Conference on Machine Learning (ICML)*, pages 449–458. PMLR, 2017.

[8] Fortunato, M., Azar, M. G., Piot, B., Menick, J., Osband, I., Graves, A., Mnih, V., Munos, R., Hassabis, D., Pietquin, O., et al. Noisy networks for exploration. In *International Conference on Learning Representations (ICLR)*, 2018.